arXiv:2611.09715v1 [cs.AI] 6 Nov 2026

Intent-Aware Caching for Efficient LLM Serving

Kazi Hasan Ibn Arif
Virginia Tech
Blacksburg, VA, USA
hasanarif@vt.edu

JinYi Yoon
Inha University
Incheon, South Korea
jinyiyoon@inha.ac.kr

Dimitrios S. Nikolopoulos
Virginia Tech
Blacksburg, VA, USA
dsn@vt.edu

Hans Vandierendonck
Queen's University Belfast
Belfast, UK
h.vandierendonck@qub.ac.uk

Deepu John
University College Dublin
Dublin, Ireland
deepu.john@ucd.ie

Bo Ji
Virginia Tech
Blacksburg, VA, USA
boji@vt.edu

Abstract

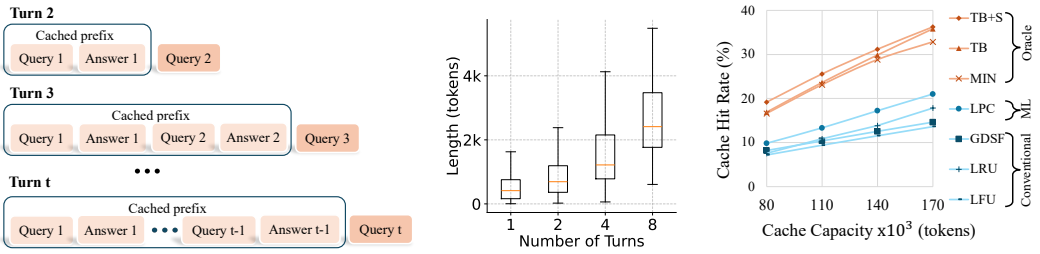
Modern LLM serving systems use prefix caching to accelerate multi-turn conversational workloads. They cache the key-value states of previous conversation prefixes and reuse them when subsequent requests share the same prefix, to avoid redundant prefill computation and reduce serving latency. However, achieving a high cache hit rate remains challenging because cache reuse depends on whether a conversation continues and how soon the next turn arrives. Our analysis of real workloads shows that user intent is a useful signal for this behavior, as different intents exhibit different continuation probabilities and inter-turn delays. Based on this key insight, we propose Intent-Aware Caching (IAC), an online eviction policy that classifies requests by intent, learns lightweight reuse statistics from recent serving logs and leverages these statistics to guide an oracle-inspired eviction strategy. Extensive evaluation on real workloads (WildChat and ShareChat) shows that IAC reaches a 34.6% cache hit rate. This substantially outperforms both the previous learning-based approach (18.8%) and traditional methods such as least recently used (LRU, 12.2%), while reducing average serving latency by up to 46.1%. Code is available at `https://github.com/hasanar1f/intent-aware-caching`.

1 Introduction

Chatbots using large language models (LLMs), such as ChatGPT [1] and Gemini [2], serve millions of users every day [3]. Users often interact with these chatbots through a sequence of follow-up turns to complete different tasks [4], commonly known as multi-turn conversations (Fig. 1a). In each turn, the request includes the past conversation history as a prefix, followed by the new user query. As a result, the input context grows with the number of turns (Fig. 1b), making inference increasingly expensive and resulting in higher time-to-first-token (TTFT) latency and lower throughput (i.e., the total number of tokens processed per second). To serve such prefix-dominated workloads efficiently, modern LLM serving systems adopt *prefix caching* [5, 6, 7]. This is an exact caching mechanism that stores the key-value (KV) states of previously processed tokens and reuses them when subsequent requests share the same prefix, regardless of conversation or user. Therefore, the server only computes the KV states for the newly added tokens instead of recomputing the entire conversation history. Prefix caching directly reduces redundant prefill computation and improves TTFT and throughput [5, 8]. Accordingly, major providers charge $10\times$ less for cached input tokens than for uncached ones [9, 10].

However, achieving a high cache hit rate (i.e., the number of input tokens whose KV states are reused from the cache, divided by the total number of input tokens over all requests) remains challenging in existing LLM serving systems, as their eviction policies are typically not designed for conversational

40th Conference on Neural Information Processing Systems (NeurIPS 2026).



(a) Prefix caching in multi-turn conv. (b) Context len. in turns (c) Online policies vs. oracles

Figure 1: (a) In a typical multi-turn conversation, each new request includes previous user queries and model responses as a prefix; caching this prefix reduces redundant prefill computation and lowers TTFT. (b) The input context grows with the number of turns, increasing serving cost. (c) Both conventional and ML-based policies achieve much lower cache hit rates than oracle policies.

workloads [11, 12], which differ significantly from traditional object caching. This is because a cached prefix is useful only when the user continues the conversation with a follow-up turn. Even when a follow-up occurs, the reuse delay can vary substantially as users read, think, or compose their next message. In addition, cache reuse is structurally dependent across turns: a hit at turn-t requires preserving the prefix formed by turns 1 through $t-1$. Cache entries also grow with context length, resulting in highly variable object sizes and making eviction decisions more sophisticated than simple policies used in traditional caching regimes [13, 14]. These challenges are further amplified by workload drift, where cache reuse patterns can shift even within an hour in practical workloads [15].

Conventional policies such as least recently used (LRU), least frequently used (LFU), and greedy dual size-frequency (GDSF) are widely used because they are simple and introduce negligible runtime overhead [11, 12, 15]. However, they rely only on generic signals such as recency, frequency, and object size. These signals do not directly capture whether a conversation will continue or when its prefix will be reused, resulting in poor eviction decisions. Recently proposed Learned Prefix Caching (LPC) trains a machine learning (ML) model to predict the likelihood of follow-up turns from conversation history [16]. However, its gains are limited because continuation probability alone does not capture cache reuse patterns (e.g., a session may continue after a long delay), making the retained prefix less useful under limited GPU memory. Moreover, LLM workloads could drift significantly within an hour [15], making frequently retrained ML predictors operationally expensive.

Offline oracles, in contrast to online policies, have access to future arrival information and therefore provide an upper-bound reference for cache performance. Belady's MIN is optimal when objects have the same size [17]. Prefix caches, however, contain KV entries with variable sizes, making the optimal offline eviction problem NP-complete [18]. Therefore, prior work on variable-sized object caching proposes practical oracle approximations such as Threshold-Belady (TB) and its size-aware variant TB+S [14]. These policies are well suited for prefix caching because cached prefixes differ in both reuse time and size. As shown in Fig. 1c, existing online policies remain substantially below oracle performance on real conversational workloads.[1] This persistent gap motivates a key question: *How to design an online cache eviction policy for conversational workloads that closes the gap to oracle performance, remains efficient at runtime, and requires low maintenance in deployment?*

In response, we propose *Intent-Aware Caching (IAC)*, an online cache eviction policy that uses workload-specific statistics to better estimate future cache reuse and guide eviction decisions. A key insight behind IAC is that a user's goal in a conversation, which we refer to as the *intent*, reveals useful *cache reuse patterns*. These cache reuse patterns include how many turns a conversation is likely to have (i.e., the number of turns) and how long users tend to wait between turns (i.e., reuse delay). Our experiments on real workloads reveal that grouping requests by intent and turn exposes meaningful patterns in cache reuse (see details in Section 3). For example, a conversation about generating code from a user instruction (i.e., `code_generation`) often involves more turns than a request for `translation`. Similarly, we often observe longer delays for `code_generation` tasks than for simple `chitchat`. Based on this observation, IAC estimates cache reuse patterns for each workload category from simple online statistics (i.e., empirical continuation frequencies and a fitted

[1] To the best of our knowledge, WildChat [19] and ShareChat [20] are the only two public datasets that contain both conversation content and timestamps, comprising a total of 1.2M conversations between real users and ChatGPT-family models [1].

delay distribution) and translates the results into an oracle-inspired eviction policy. IAC consists of three main components (see Fig. 4). First, the lightweight *intent classifier* assigns each request to a workload category. Second, the *statistics estimation* module periodically estimates statistics for each category, including the follow-up likelihood and the delay distribution from recently completed request logs. Finally, the *cache eviction policy* uses these learned statistical signals to mimic TB+S, a size-aware oracle policy, to make an eviction decision in an online fashion.

Contributions. In this work, we make three main contributions: (i) we characterize multi-turn conversational workloads through user intent from a cache-reuse perspective, showing that different intents exhibit distinct continuation and inter-turn delay patterns, which are useful for cache management; (ii) by leveraging these intent-aware cache reuse patterns, we propose IAC, an online cache eviction framework that combines lightweight statistics estimation with an oracle-inspired eviction strategy; (iii) we implement IAC and evaluate it on realistic setups, demonstrating consistent improvements in cache hit rate, TTFT, and throughput across workloads and model scales.

Evaluations. We implement IAC on SGLang [12], a widely used LLM serving framework, and evaluate it on real conversational workloads (WildChat [19] and ShareChat [20]) using LLMs from 8B to 110B on a realistic multi-GPU setup with 4×A100 (80 GB) GPUs. Across workloads, model sizes, and cache capacities, IAC consistently outperforms conventional and ML-based policies, narrowing the gap to the oracle. On ShareChat at 140K token capacity, IAC reaches 34.6% cache hit rate, compared with 12.2% for LRU and 18.8% for LPC. These cache gains translate into lower serving latency: on Qwen-1.5-110B, IAC reduces TTFT to 3.76 s, compared with 6.98 s for LRU and 5.20 s for LPC. IAC also sustains higher throughput under load: at 10 requests/s, it reduces average latency to 3.41 s, compared with 7.01 s for LRU and 5.44 s for LPC. Finally, IAC improves memory efficiency: on ShareChat with Llama-3-70B, it reaches a higher cache hit rate with only 24.4 GB of KV cache than LPC does with 51.9 GB, saving 53% of GPU memory.

2 Background and Related Work

LLM serving for multi-turn conversational workloads. LLM inference consists of two phases: *prefill* and *decoding*. Let x_t denote the input request at turn t, with a context length of L_t tokens. During prefill, the model processes all L_t input tokens through the transformer layers [21] to construct the KV states and produce the first output token. For a model with n layers and hidden dimension d, the self-attention computation in prefill has time complexity $O(nL_t^2 d)$ and KV-state memory cost $O(nL_t d)$ [21]. Therefore, the TTFT latency is highly sensitive to the input context length. This becomes particularly costly in multi-turn conversations, where each new request includes the entire conversation history as a prefix (see Fig. 1a). Specifically, a turn-t request can be written as $x_t = [u_1, y_1, \ldots, u_{t-1}, y_{t-1}, u_t]$, where u_i and y_i denote the user query and model response at turn i. Hence, $L_t = \sum_{i=1}^{t-1}(|u_i| + |y_i|) + |u_t|$, where $|\cdot|$ denotes the number of tokens in a sequence, grows with the number of turns. After prefill, decoding reuses the KV states and generates the response autoregressively, requiring one forward pass per generated token. Thus, while both phases contribute to serving cost, multi-turn conversational workloads are especially expensive because the growing conversation history repeatedly increases the prefill cost and directly inflates TTFT latency.

Prefix caching. Prefix caching is an exact caching mechanism that stores the KV states of previously processed tokens and reuses them when subsequent requests share the same token prefix, without changing model outputs [5, 7, 8]. Formally, for a turn-t request $x_t = [p_t; s_t]$ with prefix p_t of length $|p_t|$ and suffix s_t of length $L_t - |p_t|$, if the KV states of p_t are already cached, the serving system can skip recomputing the prefix and run prefill only on the remaining suffix. Modern serving systems implement this lookup efficiently using prefix-aware cache indices. For example, SGLang's RadixAttention maintains cached KV states in a radix tree, where the runtime performs longest-prefix matching over the incoming token sequence, reuses the KV nodes on the matched path, and computes only the unmatched suffix [12]. This directly reduces the prefill cost from processing L_t tokens to processing only $L_t - |p_t|$ new tokens, thereby lowering TTFT. Prefix caching is independent of user or conversation identity: a cache hit occurs whenever the token prefix matches exactly, regardless of which user or session generated the request. In practice, however, cross-user KV reuse is rare, and most cache hits come from requests submitted by the same user or the same ongoing conversation [15]. Multi-turn conversations are strongly prefix caching friendly because the shared prefix is dominated by the accumulated conversation history, e.g., $[u_1, y_1, \ldots, u_{t-1}, y_{t-1}]$, which is repeatedly prepended to subsequent turns as illustrated in Fig. 1a. Thus, although the decoding cost remains unchanged,

prefix caching substantially improves multi-turn serving by eliminating redundant prefill computation over the repeated conversation history, thereby reducing TTFT and improving serving throughput.

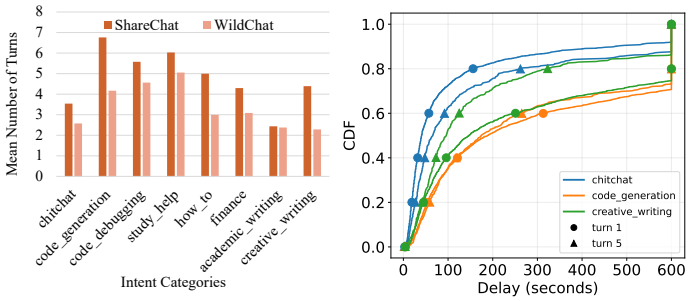
Related Work. A large body of work improves LLM serving efficiency through batching and scheduling [22, 23, 24], memory management [11], and disaggregated inference [25, 26]. Conversational workloads amplify this challenge because growing histories make prefill increasingly expensive and TTFT-critical. Prefix caching addresses this bottleneck by storing and reusing KV states when subsequent requests share the same token prefix [12, 7, 27]. In contrast, semantic caching matches requests by meaning rather than exact tokens, which can change model outputs or reduce accuracy [28, 29, 30]. Prefix caching preserves exact outputs, but its benefit is limited by GPU memory capacity. Thus, the design of cache eviction policy becomes a central challenge.

Traditional eviction policies, such as LRU, LFU, and GDSF, use generic signals including recency, frequency, and object size [13]. Learning-based policies, such as LRB [14], 3L-Cache [31], and GL-Cache [32], learn reuse patterns from past accesses in content delivery networks [33], storage systems, and object caches. While these works show the value of learned eviction, they target workloads different from multi-turn conversations and are therefore not directly applicable. A production workload study characterizes LLM serving traces and proposes a Workload-Aware (WA) eviction policy that uses request-type-level reuse estimates [15]. More recently, LPC trains an ML model to predict conversation continuation likelihood for eviction decisions [16], while T-LRU targets tail TTFT latency to improve SLO attainment [34]. Continuum sets a KV-cache time-to-live for multi-turn agents from tool-call durations [35]. Other systems extend effective cache capacity beyond GPU memory through offloading [36, 37, 38]. These approaches are complementary to our work, which focuses on prefix-cache eviction for conversational workloads under a fixed cache capacity.

3 Key Insights

The unique characteristics of conversational workloads and the large cache hit rate gap between existing online eviction policies and oracle policies motivate us to characterize this workload more closely. We hypothesize that conversational workloads contain measurable reuse patterns and that an online policy exploiting these patterns can narrow the gap to the oracle.

Existing cache eviction policies struggle with multi-turn conversational workloads. This is because cache reuse is highly heterogeneous, depending not only on whether a conversation continues, but also on how soon the next turn arrives [15, 39]. To characterize this heterogeneity, we analyze two real-world datasets with timestamps: WildChat [19] and ShareChat [20].



(a) (b)

Figure 2: **(a) Insight 1**: Mean number of turns varies across intent categories. **(b) Insight 2**: Reuse delays vary across intent categories and turns. The delay CDF, clipped at 600 s, motivates using $(intent, turn)$ as the workload category.

A natural question is which workload signal explains the differences in continuation and delay patterns. We hypothesize that these patterns depend on the user's *intent*, i.e., the task objective expressed by the request [40]. Different intents impose different interaction structures: for example, code-generation and debugging conversations often require iterative refinement, while travel-planning or information-seeking conversations may terminate after fewer turns. To test this, we assign each request to an intent category using few-shot LLM prompting, following LLM-based topic and intent discovery methods [41] (see details in Section 4.1 and Appendix C). For instance, we label a request as `code_generation` when the task is to synthesize code from a specification, as `code_debugging` when the task is to diagnose or fix code, and as `travel_planning` when the task involves constructing or refining a trip plan. This characterization reveals three key insights as follows.

Insight 1: Number of turns varies across intents. In Fig. 2a, the number of turns in conversations varies across user intents. On ShareChat [20], intents such as `code_generation` and

`code_debugging` tend to involve more turns, whereas requests such as `academic_writing` and `chitchat` often end in fewer turns. We observe the same overall pattern in WildChat [19]. This suggests that some intents naturally create more opportunities for cache reuse than others.

Insight 2: Delay varies across intents and turns. In Fig. 2b, the delay between turns varies across intents and across turns within the same intent category. For example, `code_generation` requests exhibit longer delays because users may spend time reading, testing, or modifying the generated code, whereas `chitchat` tends to have shorter delays. We also observe variations across turn indices within the same intent. For instance, `creative_writing` requests at turn 1 and turn 5 show noticeably different delay patterns. This suggests that cache reuse depends not only on intent but also on turn. We further analyze the PDFs of delay (see Appendix B.1), which shows that modeling intent alone already reveals meaningful differences in delay and that combining intent with turn reveals finer-grained reuse patterns.

Insight 3: Workload drift. In Fig. 3, the turn-count and delay patterns across intent categories can shift over the course of a day. For example, on a representative day in WildChat [19], `creative_writing` shows noticeable variation in mean turns, while `code_generation` shows variation in delay. This indicates clear workload drift over time. As a result, a static offline-trained ML predictor may not adapt well to such rapid changes without frequent retraining.

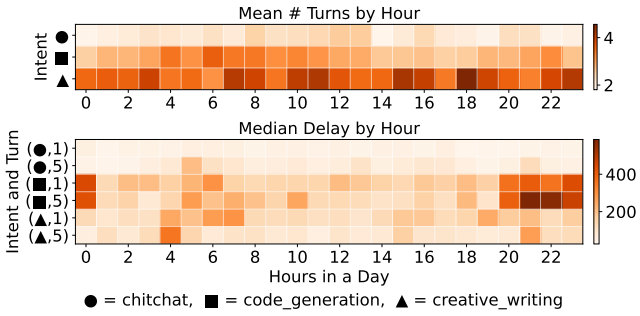


Figure 3: **Insight 3**: Cache reuse patterns (i.e., #turns and delay) vary over the course of a representative day.

Design goal. Our goal is to design an online cache eviction policy that is lightweight, adaptive, and effective under the heterogeneous reuse behavior of conversational workloads. This raises three key questions: **(Q1)** How can we identify request intent with minimal overhead? **(Q2)** How can we continuously learn reuse patterns as workloads drift? **(Q3)** How can we turn these learned patterns into an effective online cache eviction policy?

4 Our Design: Intent-Aware Caching (IAC)

By addressing the above questions, we propose *Intent-Aware Caching* (IAC), which consists of three main components (Fig. 4). First, we identify the request category using a lightweight intent classifier that is suitable for latency-sensitive serving environments (Section 4.1). Second, we periodically learn cache reuse behavior through statistics estimation so that the policy can adapt to workload drift (Section 4.2). Lastly, we incorporate the learned statistics into an oracle-inspired online eviction policy (Section 4.3).

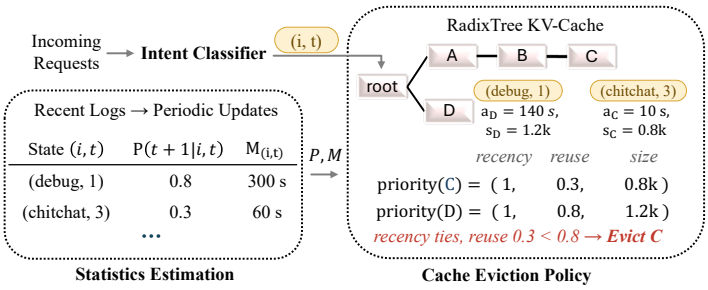


Figure 4: Overview of IAC. The **Intent Classifier** assigns an intent category to each newly arrived request. The **Statistics Estimation** module tracks the batch of recent logs and periodically updates cache reuse statistics. During serving, the **Cache Eviction Policy** (see Eq. (1)) computes a priority score for the leaf nodes (e.g., C and D in the figure) and evicts the node with the lowest score.

4.1 Intent Classifier

To classify a request by workload category at the $(intent, turn)$ level, we need to infer its intent with minimal overhead. Using an LLM or a large ML model defeats this purpose due to additional computational overhead and significantly increased latency during serving. This motivates us to design a separate lightweight intent classifier that can be trained offline once and then reused during

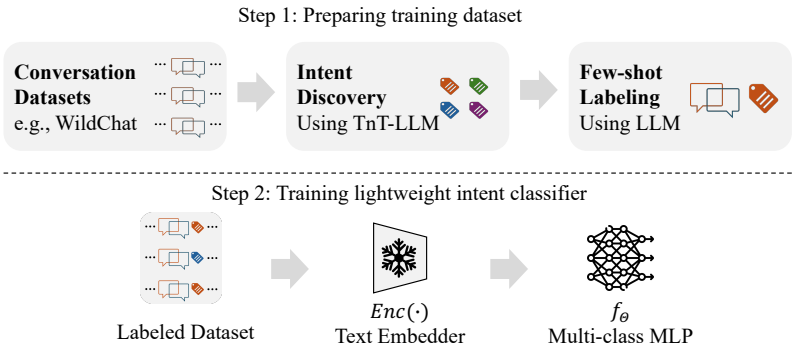


Figure 5: Offline training pipeline of the intent classifier. **Step 1**: intent discovery with TnT-LLM [41] and few-shot LLM labeling. **Step 2**: training an MLP head on a frozen text embedder.

deployment. As shown in Fig. 5, we build the classifier in two steps: we first construct a labeled dataset by discovering an intent taxonomy and labeling conversations with an LLM (Step 1), and then train a lightweight classifier on this dataset (Step 2).

Step 1: Preparing the training dataset. We sample a 200K-conversation split from WildChat [19], ShareChat [20], ShareGPT [42], and LMSys-Chat [43] to cover diverse domains and languages. This split is strictly disjoint from the online evaluation traces, avoiding data leakage between classifier training and cache-policy evaluation. We then perform *intent discovery* on this split with TnT-LLM, an LLM-based topic discovery framework [41], using the open-weight GPT-OSS-120B model [44] as the LLM backbone. Starting from the 7 classes introduced in the ShareChat paper [20], it iteratively adds classes to improve coverage. We then remove low-frequency classes and merge semantically similar ones, resulting in 30 intent categories (see Appendix C for details). Finally, using this taxonomy, we label the training conversations with few-shot LLM prompting.

Step 2: Training a lightweight intent classifier. After labeling, we train a lightweight intent classifier. The key challenge is to keep the classifier accurate while maintaining millisecond-level inference latency. We therefore pair a *frozen text embedder* Enc with a small *multi-class multilayer perceptron (MLP)* head f_θ. Since user requests span diverse domains and languages, we compare several multilingual embedding backbones by accuracy and latency (see Table 1). Formally, for a turn-t request, the classifier takes up to the last four user queries $\tilde{x}_t = [u_{\max(1,t-3)}, \ldots, u_t]$, mean-pools their embeddings, and predicts the intent $\hat{i} = \arg\max_i f_\theta(\mathrm{Pool}(\mathrm{Enc}(\tilde{x}_t)))_i$. We keep Enc frozen and train only f_θ, a three-layer MLP (embedding dim. $\to 2048 \to 1024 \to 30$) with a softmax over the 30 intent categories. We split the labeled data into training, validation, and test sets, train the head with AdamW [45] and a cosine learning-rate schedule, and select the checkpoint with the best validation macro-F1 score (see Appendix A.1 for details).

4.2 Statistics Estimation

Our analysis in Section 3 shows that cache reuse depends on both intent and turn. We therefore represent each workload state as a tuple (i, t), where i denotes the intent category and t denotes the turn index. To avoid the overhead of frequent retraining, IAC does not learn a predictive ML model online. Instead, it continuously maintains lightweight reuse statistics for each state (i, t). In particular, for every state (i, t), we estimate (i) the probability that the conversation continues to turn-$(t+1)$ and (ii) the delay distribution until that next turn arrives as follows.

Reuse probability. For a request in state (i, t), we estimate the probability of receiving a follow-up request at turn-$(t+1)$ as $P(t+1 \mid i, t) = C(i, t+1)/C(i, t)$, where $C(i, t)$ denotes the number of intent-i conversations among the N most recent conversations that reach turn-t. Since the intent is recomputed at every turn (Section 4.1), we assign the most recent intent to the whole conversation once it is completed. The window size N is a design choice that trades off stability and adaptivity: larger windows provide more stable estimates, while smaller windows adapt faster to workload drift and changing arrival dynamics (see Appendix B.3 for details).

Delay distribution. For each state (i, t), we model the wall-clock delay between turn-t and turn-$(t+1)$. Inter-turn delays are typically long-tailed and follow a log-normal distribution [39]. Therefore,

we learn $\log D_{i,t} \sim \mathcal{N}(\mu_{i,t}, \sigma^2_{i,t})$, where $\mu_{i,t}$ and $\sigma^2_{i,t}$ are the mean and variance of the log-delay, respectively. Intuitively, mean $\mu_{i,t}$ captures the typical delay of state (i,t), while the variance $\sigma^2_{i,t}$ captures how spread out or variable the delays are. A small $\mu_{i,t}$ indicates short reuse intervals, and a small $\sigma_{i,t}$ indicates predictable ones, allowing the eviction policy to distinguish such states from those whose reuse intervals are long or unpredictable.

4.3 Cache Eviction Policy

To translate the learned statistics into an online cache eviction rule, we design IAC to mimic the behavior of TB+S [14], which achieves the best oracle performance on our conversational workloads (Fig. 1c). Formally, the oracle priority is $(\mathbf{1}[\Delta\text{next_arrival} < \text{Threshold}], \text{size})$, where $\mathbf{1}[\cdot]$ is the indicator function, size is the number of tokens, and Δnext_arrival is the time until the next reuse (set to infinity when no future reuse occurs). Intuitively, entries that will be reused soon should remain in cache; among entries with the same reuse status, larger entries are preferred for more reusable tokens.

To mimic TB+S's decision online, IAC uses the statistics learned for each workload state (i,t), in the same order as the oracle priority. First, the recency signal is derived from the inter-turn delay distribution of the state (i,t), which is modeled using its learned parameters. Since the delay is long-tailed, we summarize its central reuse timescale by the median delay obtained from the log-normal model as $M_{i,t} = \exp(\mu_{i,t})$. We use this median as the estimated reuse window: if a cached item is younger than $M_{i,t}$, it is considered to be still within its likely reuse period; otherwise, its chance of near-term reuse is lower. We find the median recency estimate to work better than alternatives such as the mean or mode (Fig. 9). Second, after this recency test, IAC uses the reuse signal $P(t+1 \mid i,t)$ estimated in Section 4.2, i.e., the probability that a request in state (i,t) receives a follow-up turn, as the next ordering signal. Finally, it uses size as the last tie-breaker, following TB+S. Formally, consider a leaf node v in the RadixTree, created by a request with intent i_v at turn t_v. Let a_v and s_v be the age (the time since it was cached) and the size (the number of cached tokens) of node v, respectively. Nodes are compared lexicographically (i.e., recency first, followed by reuse, and finally size) by the following priority, and the node with the lowest priority is evicted first:

$$\text{priority}(v) = \big(\underbrace{\mathbf{1}[a_v < M_{i_v,t_v}]}_{\text{Recency signal}},\ \underbrace{P(t_v+1 \mid i_v, t_v)}_{\text{Reuse signal}},\ s_v\big). \tag{1}$$

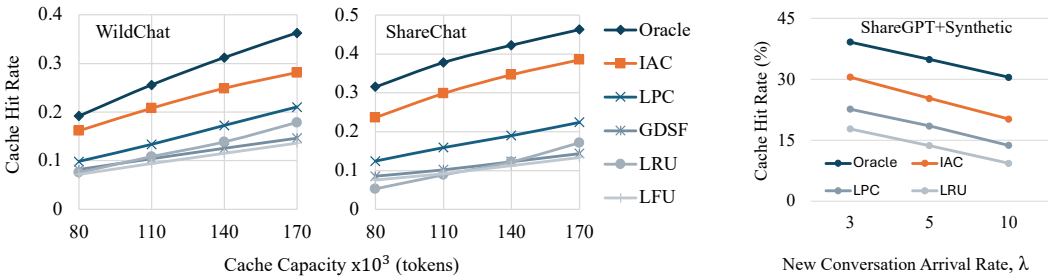
5 Evaluation

We evaluate IAC and the baselines along four dimensions: cache hit rate (Section 5.1), TTFT latency (Section 5.2), throughput under load (Section 5.3), and ablation studies (Section 5.4).

Baselines. We compare IAC with conventional eviction policies (LRU, LFU, and GDSF) and the ML-based LPC policy. In addition, we include TB+S, an offline oracle that knows the next arrival time of every cached entry and therefore serves as an upper-bound reference rather than a deployable policy. In the main comparison, we do not include WA [15], as it relies on reuse statistics over four request types (text, multimodal, file, and search), whereas our primary workloads contain only text requests. Instead, we compare IAC with WA using the cloud trace released by the authors, which includes request-type information (see Appendix B.5). Additionally, we consider a variant of WA adapted for our workloads and evaluate it on these workloads (see Appendix B.5).

Implementation and testbed. We implement IAC in SGLang [46] because of its modular design and wide adoption (see Appendix A.2 for details and Appendix A.3 for the baselines). Our experiments run on a cloud-hosted multi-GPU node with an Intel Xeon Platinum 8380 CPU, 40 CPU cores, and four NVIDIA A100 (80 GB) GPUs, connected through PCIe 4.0 $\times$16 links. To evaluate IAC across model scales and deployment settings, we use five open-weight LLMs ranging from 8B to 110B parameters [47, 48, 49], which are served on single-, two-, and four-GPU configurations.

Workloads. We primarily evaluate IAC on WildChat [19] and ShareChat [20], since both provide original timestamps. Additionally, we use LMSys-Chat [43] and ShareGPT [42], which do not include timestamps, to construct synthetic timestamped workloads and study robustness under different arrival patterns (Section 5.1). Finally, we use a production trace from a major cloud provider to compare against WA, since this trace includes the request-type information required by WA [15]. For each dataset, after sorting by timestamp, we use the first 40K conversations for LPC training and statistics estimation and use the following 10K conversations to construct the test workload. We provide details of dataset statistics and workload construction in Table 2 and Appendix A.4, respectively.



(a) Cache hit rate across various capacities

(b) Across arrival rates

Figure 6: Cache hit rate of IAC vs. the baselines. (**a**) On WildChat and ShareChat, two real workloads with original timestamps, IAC achieves the highest cache hit rate among online policies at every cache capacity. (**b**) On ShareGPT with synthetic Poisson arrival rates λ (new conversations per sec.), IAC consistently outperforms the online baselines even as higher λ increases cache contention.

5.1 Cache Hit Rate

Fig. 6a shows the cache hit rate on WildChat and ShareChat, two real multi-turn workloads with original timestamps. Across all cache capacities, IAC consistently outperforms both conventional and ML-based baselines. At 140K token capacity (43 GB for Llama-3-70B) on WildChat, LRU achieves 13.8%, LPC achieves 17.1%, and IAC achieves 24.8% cache hit rate. This corresponds to a 79.7% improvement over LRU and a 45.0% improvement over LPC. The gain is larger on ShareChat. At the same capacity, LRU achieves 12.2%, LPC achieves 18.8%, and IAC achieves 34.6%. This corresponds to a 183.6% improvement over LRU and an 84.0% improvement over LPC. These results show that IAC substantially improves cache reuse on real conversational workloads.

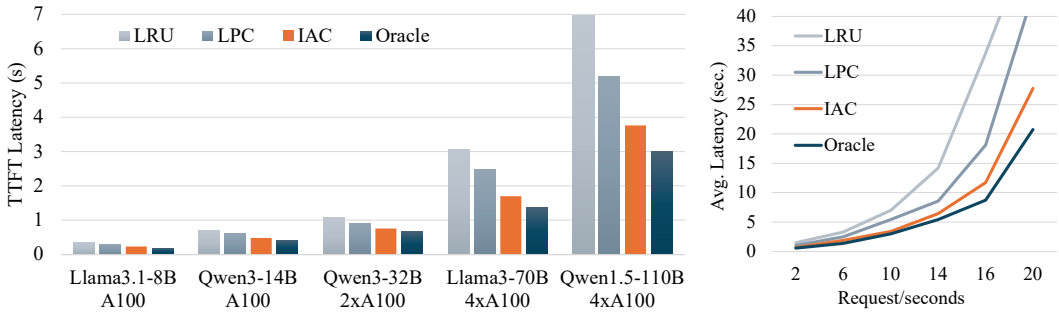
IAC also improves memory efficiency. On ShareChat with Llama-3-70B, IAC reaches a 23.7% cache hit rate with only 24.4 GB of KV cache (80K tokens), whereas LPC reaches only 22.4% even with 51.9 GB (170K tokens). This reduces GPU memory demand by 27.5 GB, corresponding to a 53% memory saving. We further evaluate IAC on ShareGPT (Fig. 6b) and LMSys-Chat (Appendix B.6) under various arrival patterns while keeping the capacity fixed (150K). For these workloads, new conversations arrive following a Poisson process with rate λ, while inter-turn delays are sampled from a fixed long-tailed distribution with mean 4.6 and variance 1, following prior workload studies [39]. The results show that IAC outperforms the baselines and remains robust under various arrival patterns.

5.2 Time-to-First-Token (TTFT) Latency

Fig. 7a compares mean TTFT across different LLMs and multi-GPU setups on ShareChat [20], using a request rate of 5 req/s and a fixed cache capacity of 150K tokens. IAC consistently achieves a lower TTFT than both conventional and ML-based policies, with larger gains on bigger models where prefill latency dominates. For example, on Llama-3-70B, IAC reduces TTFT to 1.7 s, compared with 3.1 s for LRU, 2.5 s for LPC, and 1.4 s for the oracle. On Qwen-1.5-110B, IAC achieves 3.76 s, compared with 6.98 s for LRU, 5.20 s for LPC, and 3.01 s for the oracle. Overall, IAC reduces TTFT by up to 46.1% over LRU and up to 32.0% over LPC, showing that its cache-hit gains translate into substantial latency reductions on real multi-turn workloads. We also report the tail latency in Appendix B.8.

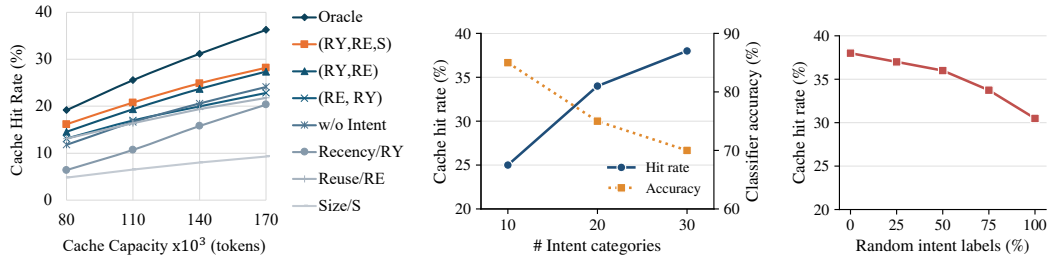
5.3 Throughput

Fig. 7b reports the average serving latency on Llama-3-70B (4×A100). As the load grows, queuing delay increases the average latency of all policies. However, the latency of IAC grows more slowly, because higher cache reuse removes redundant prefill computation and frees GPU time for other requests. For example, at 10 req/s, IAC achieves 3.41 s latency, compared with 7.01 s for LRU and 5.44 s for LPC, reducing latency by 51.4% and 37.3%, respectively. Under heavier load at 20 req/s, IAC reduces latency by 49.2% over LRU and 36.6% over LPC. Overall, averaged across request rates, IAC reduces serving latency by over 50% compared with LRU and around 35% compared with LPC. Equivalently, for a given latency target, IAC can sustain a higher request rate.



(a) TTFT Latency (b) Throughput

Figure 7: (**a**) IAC reduces TTFT across various models by improving cache hit rate. (**b**) IAC maintains lower average latency as request rate increases, showing higher throughput under load.



(a) Eviction-policy ablation (b) Intent granularity (c) Classifier label noise

Figure 8: Ablation and sensitivity of IAC. (**a**) Intent-turn grouping with the recency, reuse, and size signals performs best. (**b**) Finer intent categories lower classifier accuracy but raise cache hit rate by preserving reuse differences that coarser groups merge. (**c**) Cache hit rate degrades gracefully as intent labels are replaced with random ones, since the turn, recency, and size signals remain intact.

5.4 Ablation Study

How much does each IAC signal contribute, and does the ordering matter? As shown in Fig. 8a, we ablate the three signals used by IAC: recency (RY), reuse probability (RE), and size (S). Each signal improves cache hit rate, but reuse is stronger than recency when used alone, showing that follow-up likelihood is the stronger individual predictor of prefix reuse. Combining RY and RE improves performance further, and the ordering matters: $(\mathrm{RY}, \mathrm{RE})$ consistently outperforms $(\mathrm{RE}, \mathrm{RY})$. Adding size as the final tie-breaker gives $(\mathrm{RY}, \mathrm{RE}, \mathrm{S})$, which achieves the best performance among all online variants and remains closest to the oracle across cache capacities.

Does intent provide additional value beyond turn? To isolate the role of intent, we remove intent from the workload category and group requests only by turn index (referred to as *w/o Intent*) in Fig. 8a. This reduces the cache hit rate, showing that the turn index alone is not sufficient. Combining intent and turn creates more effective groups for learning conversational reuse.

How many intent categories are needed? We study the trade-off between performance and intent granularity in Fig. 8b. Our TnT-LLM [41]-based intent discovery produces a 30-category taxonomy. To test whether fewer categories are sufficient, we merge it into coarser 20-category and 10-category variants. We provide the full merge lists in Appendices C.2 and C.3. Coarser grouping improves classifier accuracy because the classification task becomes easier. However, it also removes behavioral distinctions that matter for caching. As a result, the 10-category variant achieves higher classifier accuracy but much lower cache hit rate, while the 30-category taxonomy gives the best cache performance. This shows that preserving fine-grained reuse structure is more important than maximizing classifier accuracy alone.

How accurate does the intent classifier need to be? To test sensitivity to classifier errors, we randomly replace a fraction of predicted intent labels at serving time and measure the cache hit rate at a fixed cache capacity (170K tokens) on ShareChat, presented in Fig. 8c. We vary the fraction of randomized intent labels from 0% to 100%. Results show that IAC degrades gracefully as classifier errors increase.

Even with fully random labels, IAC still achieves a 30% cache hit rate because randomizing the intent removes only the intent signal while the other signals remain intact, and each random-intent group becomes a random sample of all conversations at turn t, so its statistics approach turn-level averages. This indicates that IAC does not require a perfect classifier. Instead, it only needs intent predictions to be accurate enough to preserve coarse reuse differences across workload groups.

Why do we use the median delay as the reuse window? We compare three estimators of the reuse window $M_{i,t}$ derived from the learned log-normal delay model: the mean $e^{\mu_{i,t}+\sigma_{i,t}^2/2}$, the median $e^{\mu_{i,t}}$, and the mode $e^{\mu_{i,t}-\sigma_{i,t}^2}$. As shown in Fig. 9, the median provides the best performance. This is because inter-turn delays in conversational workloads are typically long-tailed (Fig. 2b), where a small fraction of very large delays skews the distribution. The mean is sensitive to these outliers and overestimates the reuse window, causing the policy to retain stale entries longer than useful. The mode, on the other hand, captures only the most frequent short-delay region and underestimates the reuse window, resulting in premature eviction of reusable prefixes. In contrast, the median provides a robust estimate of the typical reuse timescale, balancing early and late reuse, and therefore yields a higher cache hit rate.

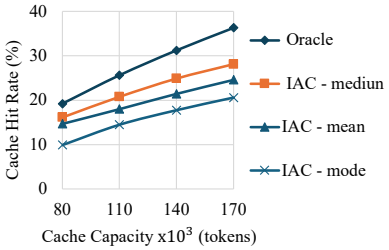


Figure 9: Reuse-window estimators.

Which embedding model is sufficient for intent classification? We compare embedding backbones by classifier accuracy and inference latency in Table 1. The E5 family [50] consistently outperforms ModernBERT [51] while maintaining low latency, making it a better fit for online serving. This is likely because E5 models are trained for multilingual text embedding, which better matches real chatbot workloads with diverse user queries. Among the E5 variants, the instruction-tuned E5-Large-Inst achieves the best accuracy, reaching 75% with 20 ms latency. Therefore, we choose E5-Large-Inst as the backbone in IAC.

Table 1: Embedding backbone models.

Model (size)	Acc.	Lat.
ModernBERT (0.2B+3.7M)	62%	33 ms
E5-Small (0.1B+2.9M)	70%	13 ms
E5-Base (0.2B+3.7M)	71%	16 ms
E5-Large (0.5B+4.2M)	72%	20 ms
E5-Large-Inst (0.5B+4.2M)	75%	20 ms

6 Conclusion

This work studies prefix-cache eviction for multi-turn conversational workloads. Our key observation is that cache reuse is highly heterogeneous: different requests vary in both reuse likelihood and inter-turn delay. IAC exploits this heterogeneity by grouping requests based on intent and turn index, and then learning lightweight reuse statistics within each group. The outcome of this study suggests that practical prefix-cache eviction should move beyond generic signals and exploit workload-aware signals that capture how users interact with LLM systems.

While this paper advances prefix-cache eviction for multi-turn conversational workloads, there are still several limitations. First, we evaluate IAC mainly on conversational workloads, where cache reuse is driven by repeated multi-turn context. However, prefix caching can also benefit other workloads with repeated prefixes, such as agentic workflows, tool-use pipelines, and API applications with shared system prompts or templates. Second, our intent classifier keeps the embedding backbone frozen and trains only a lightweight MLP prediction head. The ablation in Table 1 suggests that stronger or task-adapted embeddings may improve intent classification, which could further improve cache eviction decisions and cache hit rate. Future work can consider extensions that include broader workload signals and broader LLM serving settings. Furthermore, workload-aware signals, such as intent, turn structure, and tool usage, could be used beyond cache eviction to jointly optimize batching, scheduling, and routing decisions across model fleets.

Acknowledgments

This work was supported in part by the National Science Foundation under Grant CNS-2315851, the Commonwealth Cyber Initiative, the Virginia Tech College of Engineering, the Virginia Tech Institute for Advanced Computing, a research grant from the Department for the Economy, Northern Ireland, under the US-Ireland R&D Partnership Programme grant agreement USI-226, Taighde Éireann – Research Ireland (22/US/3848) under the US-Ireland R&D Partnership Programme, and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2026-25602748).

References

[1] OpenAI. ChatGPT. `https://chatgpt.com`, 2026. Conversational AI based on large language models.

[2] Google. Gemini. `https://gemini.google.com`, 2026. Conversational AI based on large language models.

[3] Aaron Chatterji, Thomas Cunningham, David J Deming, Zoe Hitzig, Christopher Ong, Carl Yan Shan, and Kevin Wadman. How people use chatgpt. Technical report, National Bureau of Economic Research, 2025.

[4] Qifu Wen, Prishita Kochhar, Sherif Zeyada, Tahereh Javaheri, and Reza Rawassizadeh. From clicks to conversations: Evaluating the effectiveness of conversational agents in statistical analysis. *International Journal of Human–Computer Interaction*, pages 1–18, 2025.

[5] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In P. Gibbons, G. Pekhimenko, and C. De Sa, editors, *Proceedings of Machine Learning and Systems*, volume 6, pages 325–338, 2024.

[6] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with cachedattention. In *Proceedings of the 2024 USENIX Conference on Usenix Annual Technical Conference*, USENIX ATC'24, USA, 2024. USENIX Association.

[7] OpenAI. https://developers.openai.com/api/docs/guides/prompt-caching, 2026. Accessed on Mar 8, 2026.

[8] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-Efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, Santa Clara, CA, July 2024. USENIX Association.

[9] Anthropic. https://platform.claude.com/docs/en/about-claude/pricing, 2026. Accessed on Mar 8, 2026.

[10] OpenAI. https://openai.com/api/pricing/, 2026. Accessed on Mar 8, 2026.

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *SOSP '23*, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery.

[12] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

[13] Ludmila Cherkasova. *Improving WWW proxies performance with greedy-dual-size-frequency caching policy*. Hewlett-Packard Laboratories Palo Alto, CA, USA, 1998.

[14] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. Learning relaxed belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 529–544, Santa Clara, CA, February 2020. USENIX Association.

[15] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. {KVCache} cache in the wild: Characterizing and optimizing {KVCache} cache at a large cloud provider. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 465–482, 2025.

[16] Dongsheng Yang, Austin Li, Kai Li, and Wyatt Lloyd. Learned prefix caching for efficient llm inference. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[17] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems journal*, 5(2):78–101, 1966.

[18] Marek Chrobak, Gerhard J Woeginger, Kazuhisa Makino, and Haifeng Xu. Caching is hard—even in the fault model. *Algorithmica*, 63(4):781–794, 2012.

[19] Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. Wildchat: 1m chatGPT interaction logs in the wild. In *The Twelfth International Conference on Learning Representations*, 2024.

[20] Yueru Yan, Tuc Nguyen, Bo Su, Melissa Lieffers, and Thai Le. Sharechat: A dataset of chatbot conversations in the wild. *arXiv preprint arXiv:2512.17843*, 2025.

[21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[22] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, July 2022. USENIX Association.

[23] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, pages 117–134, 2024.

[24] Yuhang Li, Rong Gu, Chengying Huan, Zhibin Wang, Renjie Yao, Chen Tian, and Guihai Chen. Hotprefix: Hotness-aware kv cache scheduling for efficient prefix sharing in llm inference systems. *Proc. ACM Manag. Data*, 3(4), September 2025.

[25] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.

[26] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024.

[27] Gregory Dexter, Shao Tang, Ata Fatahibaarzi, Qingquan Song, Tejas Dharamsi, and Aman Gupta. LLM query scheduling with prefix reuse and latency constraints. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[28] Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In Liling Tan, Dmitrijs Milajevs, Geeticka Chauhan, Jeremy Gwinnup, and Elijah Rippeth, editors, *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, pages 212–218, Singapore, December 2023. Association for Computational Linguistics.

[29] Waris Gill, Mohamed Elidrisi, Pallavi Kalapatapu, Ammar Ahmed, Ali Anwar, and Muhammad Ali Gulzar. Meancache: User-centric semantic caching for llm web services. In *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1298–1310. IEEE, 2025.

[30] Xutong Liu, Baran Atalar, Xiangxiang Dai, Jinhang Zuo, Siwei Wang, John C.S. Lui, Wei Chen, and Carlee Joe-Wong. Semantic caching for low-cost llm serving: From offline learning to online adaptation. In *IEEE International Conference on Computer Communications (INFOCOM)*, May 2026.

[31] Wenbin Zhou, Zhixiong Niu, Yongqiang Xiong, Juan Fang, and Qian Wang. {3L-Cache}: Low overhead and precise learning-based eviction policy for caches. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 237–254, 2025.

[32] Juncheng Yang, Ziming Mao, Yao Yue, and KV Rashmi. {GL-Cache}: Group-level learning for efficient and high-performance caching. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*, pages 115–134, 2023.

[33] Stefan Saroiu, Krishna P Gummadi, Richard J Dunn, Steven D Gribble, and Henry M Levy. An analysis of internet content delivery systems. *ACM SIGOPS Operating Systems Review*, 36(SI):315–327, 2002.

[34] Wenxin Zhang, Yueying Li, Ciamac C. Moallemi, and Tianyi Peng. Tail-optimized caching for LLM inference. In *NeurIPS 2025 Workshop MLxOR: Mathematical Foundations and Operational Integration of Machine Learning for Uncertainty-Aware Decision-Making*, 2025.

[35] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, et al. Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live. *arXiv preprint arXiv:2511.02230*, 2025.

[36] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56, 2024.

[37] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, page 94–109, 2025.

[38] Yihua Cheng, Yuhan Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. Lmcache: An efficient KV cache layer for enterprise-scale LLM inference. *CoRR*, abs/2510.09665, 2025.

[39] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. {ServeGen}: Workload characterization and generation of large language model serving in production. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 1845–1859, 2026.

[40] Galo Castillo-López, Gael de Chalendar, and Nasredine Semmar. Intent recognition and out-of-scope detection using LLMs in multi-party conversations. In Frédéric Béchet, Fabrice Lefèvre, Nicholas Asher, Seokhwan Kim, and Teva Merlin, editors, *Proceedings of the 26th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 504–512, Avignon, France, August 2025. Association for Computational Linguistics.

[41] Mengting Wan, Tara Safavi, Sujay Kumar Jauhar, Yujin Kim, Scott Counts, Jennifer Neville, Siddharth Suri, Chirag Shah, Ryen W. White, Longqi Yang, Reid Andersen, Georg Buscher, Dhruv Joshi, and Nagu Rangan. Tnt-llm: Text mining at scale with large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '24, page 5836–5847, New York, NY, USA, 2024. Association for Computing Machinery.

[42] anon8231489123. Sharegpt dataset in huggingface, 2023.

[43] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P Xing, et al. Lmsys-chat-1m: A large-scale real-world llm conversation dataset. *arXiv preprint arXiv:2309.11998*, 2023.

[44] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

[45] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[46] SGLang Project. SGLang: A fast serving framework for large language models and vision language models. `https://github.com/sgl-project/sglang/tree/v0.5.3`, 2024. GitHub repository, version v0.5.3, Apache-2.0 license, accessed April 20, 2026.

[47] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[48] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[49] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

[50] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual e5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*, 2024.

[51] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Griffin Thomas Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2526–2547, Vienna, Austria, July 2025. Association for Computational Linguistics.

[52] Yangdsh. LPC: Learned LLM Prefix Cache. `https://github.com/yangdsh/LPC`, 2025. GitHub repository. Accessed: 2026-05-05.

A Implementation and Experiment Details

A.1 Training Intent Classifier

Elaborating on Section 4.1, we train the intent classifier using the labeled conversations from the offline training dataset. For each conversation, we use up to the first four turns as input. At serving, the classifier uses up to the last four turns of the current conversation (e.g., turn 5 uses turns 2–5), following LPC [16]. Each turn is marked with a turn index, and the full input is truncated to a maximum length of 1024 tokens. We add the query prefix used by the embedding model and apply head-tail truncation within each turn to preserve both the beginning and ending context of long requests.

We use a frozen embedding backbone to avoid expensive end-to-end fine-tuning. Given the tokenized input, the embedding model produces hidden states, which are mean-pooled over non-padding tokens and then ℓ_2-normalized. These normalized embeddings are precomputed and cached before training the classifier head. This makes classifier training lightweight and isolates the cost of the embedding model from the MLP optimization.

The classifier head is a small MLP with two hidden layers: LayerNorm $\rightarrow$ Linear$(d_{\text{emb}}, 2048)$ $\rightarrow$ GELU $\rightarrow$ Dropout $\rightarrow$ Linear$(2048, 1024)$ $\rightarrow$ GELU $\rightarrow$ Dropout $\rightarrow$ Linear$(1024, K)$, where K is the number of intent classes and d_{emb} is the embedding dimension, e.g., 1024 for E5-Large-Inst. In the final system, we use $K = 30$.

We split the labeled data into training, validation, and test sets using an 80/10/10 stratified split. The MLP is trained with cross-entropy loss using AdamW with learning rate 3×10^{-3}, weight decay 0.01, dropout 0.25, and a cosine learning-rate schedule with 5% warmup. We train for up to 25 epochs and select the checkpoint with the best validation macro-F1 score. Final classifier accuracy is reported on the held-out test set.

A.2 Implementation of IAC

We implement IAC on top of SGLang (v0.5.3), an open-source LLM serving system with a modular runtime. SGLang manages its KV cache using a RadixTree, where each node stores a cached token-prefix segment and shared prefixes are stored only once. This structure naturally supports prefix reuse and makes SGLang a suitable platform for implementing prefix-cache eviction policies.

We extend each RadixTree node with the metadata required by IAC: the predicted intent label, turn index, creation timestamp, and cached-prefix size. When a node is created, IAC assigns its intent label using the request conversation context,

$$\texttt{node.intent} = \texttt{IntentClassifier}(\texttt{Req.conversation}).$$

During eviction, SGLang considers leaf nodes as eviction candidates. IAC replaces the default priority with our intent-aware priority score. For each candidate node, we compute its age as

$$\texttt{node.age} = \texttt{system.clock} - \texttt{node.timestamp},$$

and then compute the priority from the node's intent, turn index, age, and size using Eq. (1). Nodes with lower priority are evicted first. This integration keeps eviction lightweight and fits directly into SGLang's existing heap-based eviction pipeline.

To support statistics estimation, IAC maintains lightweight request logs. Each record stores the session ID, request ID, turn index, timestamp, and intent label. If a request does not receive a follow-up within 30 minutes, we treat the corresponding state as completed. IAC then estimates continuation probability and delay statistics for each intent–turn state from the recent-logs. These updates require only simple counts and delay summaries, so the update cost remains small.

To handle sparse workload states, IAC uses default statistics when empirical estimates are unreliable. If a state (i, t) is unobserved in the recent-log window, or if $C(i, t) = 0$, we do not compute $C(i, t+1)/C(i, t)$. Instead, we set $P(t+1 \mid i, t) = 0.5$ and use a default median reuse window of $150\,\text{s}$. These defaults avoid division-by-zero errors and assign a neutral priority to rare intents or deep conversational turns. Once sufficient observations are available, IAC replaces the defaults with the empirical continuation probability and learned delay statistics for that state.

Table 2: Summary of the datasets used in this study. We use ShareChat and WildChat as the main workloads because they provide original user timestamps. LMSys-Chat and ShareGPT are used to evaluate different request arrival patterns, while all four datasets are used to train the intent classifier.

Dataset	Timestamp	#Samples	Avg. length	Avg. turns	#Languages
ShareChat [20]	✓	142,808	1250	4.62	101
WildChat [19]	✓	1,039,785	736	2.54	68
LMSys-Chat [43]	✗	1,000,000	284	2.02	65
ShareGPT [42]	✗	94,145	442	3.51	41

A.3 Implementation of Baseline

Learned Prefix Caching (LPC) policy. We implement Learned Prefix Caching (LPC) [16] following the authors' open-source implementation and training recipe [52]. LPC trains a predictor that takes the conversation history as input and estimates the probability that the conversation will continue. Following the LPC design, we use the most recent user prompts, i.e., the current prompt and up to four preceding prompts, as input context, keep the embedding backbone frozen, and train only the MLP predictor for each workload. During eviction, we use the LPC priority score with its exponential time-decay function, which lowers the predicted continuation probability as the cached prefix becomes inactive. LPC is trained separately for each workload using the common training split described in Appendix A.4. We select the best checkpoint based on validation macro-F1 and evaluate LPC on the same replay trace used for all policies.

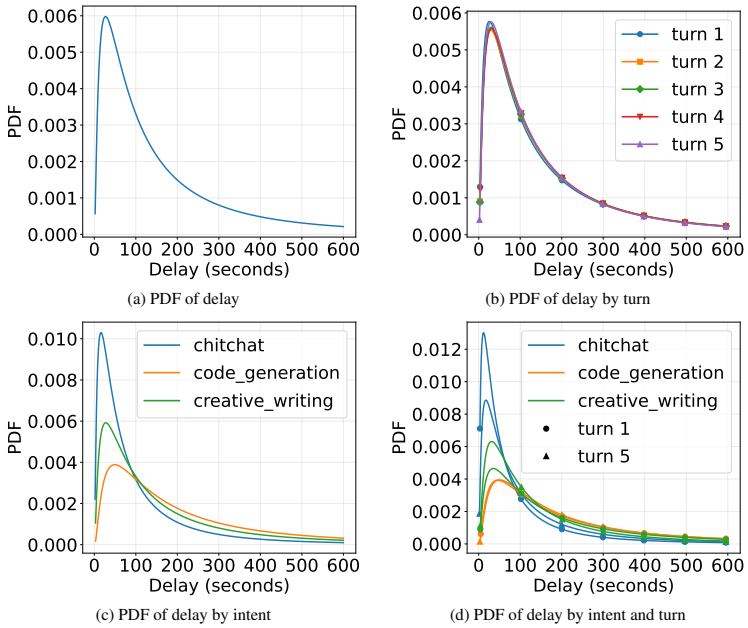
Workload-Aware (WA) policy. We adapt WA to SGLang's RadixTree-based prefix cache. In vLLM-style block caches, the offset term prevents earlier prefix blocks from being evicted before later dependent blocks. In SGLang, eviction candidates are RadixTree leaf nodes, and prefix dependencies are already preserved because internal prefix nodes are not evicted before their descendants. Therefore, we set $\texttt{Offset} = 0$ for all candidates and use the workload-specific `ReuseProb` as the WA priority. WA is trained and evaluated using the common split described in Appendix A.4.

A.4 Experiment Details

Workload split. Table 2 presents the summary of the datasets used in this study. To ensure fair data access across learned policies, we use the same history and replay split for all methods. For WildChat and ShareChat, we sort conversations by timestamp. For LMSys-Chat and ShareGPT, which do not provide original timestamps, we use the original dataset order. For each workload, the first 40,000 conversations are used as the history/training split, and the next 10,000 conversations are used as the replay trace.

Training and history window. IAC is designed to update lightweight statistics from completed request logs during deployment. However, in the main evaluation, we use the 40,000-conversation training split as the recent-log window for IAC. This matches the information available to the offline learned baselines and avoids giving IAC additional information from the replay trace. WA and LPC are also trained using the same 40,000-conversation split. Thus, all learned methods start from the same historical information and are evaluated on the same future 10,000-conversation trace.

Replay setting. During replay, all policies use the same request order, cache capacity, model, hardware setup, and arrival process. For WildChat and ShareChat, which provide real timestamps, we preserve the original inter-turn delays within each conversation. We also preserve the original conversation order in the workload, but control the relative arrival time between new conversations using a Poisson process at the target arrival rate. This allows us to match the offered load to our testbed while retaining realistic timing within each conversation. For example, replaying 10,000 new conversations at an arrival rate of 5 new conversations/s takes approximately 2,000 seconds, or 33.3 minutes. Since each conversation has multiple turns, e.g., roughly 3 turns on average, this corresponds to about 30,000 total requests and an average request rate of approximately $5 \times 3 = 15$ requests/s. Follow-up turns are scheduled according to the original inter-turn delays. For synthetic workloads from LMSys-Chat and ShareGPT, which do not provide timestamps, we sample inter-turn delays from a long-tailed delay distribution following prior workload studies [39, 15].



(a) PDF of delay

(b) PDF of delay by turn

(c) PDF of delay by intent

(d) PDF of delay by intent and turn

Figure 10: Inter-turn delay distributions on WildChat-1M [19]. Progressively finer grouping by workload signals exposes cache-reuse structure that is hidden by a single global distribution, motivating our design. Although turn index alone provides limited separation for delay modeling, it remains useful for estimating reuse probability.

B Additional Experiments

B.1 Inter-turn delay distributions

Fig. 10 further illustrates how delay patterns become more structured under finer workload groupings. When all requests are aggregated together, the delay distribution appears broad and smooth. Grouping by turn alone reveals only limited variation. In contrast, grouping by intent exposes clear differences in delay behavior across workload types, and combining intent with turn reveals even finer-grained patterns. This further supports our design choice of modeling cache reuse at the $(intent, turn)$ level.

B.2 Overhead of assigning the intent

The intent classifier assigns the intent label as soon as a request arrives in the scheduler, while the request waits in the queue. Therefore, the classification overlaps with queuing, and its worst-case overhead on the critical path is the 20 ms classifier latency when there is no queuing delay.

Table 3 shows that batching can amortize the cost. At 100 requests/s, a single instance needs only about 0.2 ms per request at a batch size of 100 and the peak GPU memory of the classifier stays 2.29 GB across batch sizes.

Table 3: Latency, throughput, and memory of the E5-Large-Inst intent classifier with batching.

Batch size	p50 (ms)	p95 (ms)	Per-request (ms)	Throughput (req/s)	Peak GPU mem. (GB)
1	19.4	20.6	19.4	52	2.29
10	19.4	19.8	1.94	515	2.29
100	19.6	19.9	0.20	5,112	2.29

B.3 Online Serving with IAC

In the main evaluation, IAC estimates its statistics once from the 40K-conversation history split and is then evaluated on the following 10K conversations (Appendix A.4). In deployment, however,

IAC is meant to run online: it periodically re-estimates its statistics from recent request logs as the workload drifts (Section 4.2). To check that IAC works in this setting, we run a causal rolling-window experiment that mimics online serving.

We split the WildChat stream into sequential 10K-conversation folds and use a cache capacity of 140K tokens. The first fold (0–10K) provides the initial statistics, and the next four folds (10K–50K) are replayed in order. Before each fold, IAC re-estimates the (i, t) statistics using only past folds, with a window of $N = 10$K conversations, so no future information is used. As shown in Table 4, IAC keeps a clear advantage over LRU in every fold (+49% to +83%), even though the statistics are refreshed from a much smaller window than in the main evaluation.

Updating the statistics requires no separate effort. The inputs (i.e., intent label, turn index, and arrival time of each request) are already recorded in the request logs as requests are served, so no extra data collection or labeling is needed. Each update is a single pass that counts continuations and fits the log-normal delay parameters for every (i, t) state, with no model training. It takes about 1.4 s on CPU and runs off the serving path, so IAC can refresh its statistics frequently at negligible cost, unlike ML-based policies that must be retrained to track drift.

Table 4: Causal rolling-window evaluation on WildChat (140K token capacity, $N = 10$K). The statistics are re-estimated from past folds only. Cache hit rate in %.

Replay window	LRU	IAC (ours)	Gain	Update time (CPU)
10K–20K	13.7	20.4	+49%	1.32 s
20K–30K	9.8	17.9	+83%	1.41 s
30K–40K	14.2	23.5	+66%	1.37 s
40K–50K	11.6	19.7	+70%	1.53 s

B.4 Using the Full Delay Distribution

We test a variant, called Full-Horizon, that replaces the binary recency test $\mathbf{1}[\text{age} < M_{i,t}]$ with a continuous score, i.e., the probability that the next turn arrives within the next H seconds given the current age of the entry, computed from the learned log-normal delay model. Full-Horizon uses this probability as the primary signal. As shown in Table 5, Full-Horizon stays below IAC at every capacity on ShareChat for two reasons. First, IAC keeps a binary recency threshold like the TB+S oracle it mimics, so recency, reuse probability, and size act in a clean order. A continuous recency score is almost unique per entry, so it dominates the ranking, and the reuse-probability and size signals rarely take effect. Second, Full-Horizon relies on the full delay distribution, including its spread and tail, which is noisy per (i, t) state, whereas the median is robust. This also matches Fig. 9, where the median outperforms the mean and the mode.

Table 5: Cache hit rate (%) on ShareChat with the binary median-based recency test (IAC) and with the full delay distribution (Full-Horizon).

Capacity ($\times 10^3$ tokens)	Full-Horizon	IAC (ours)
80	21.5	23.7
110	27.4	29.8
140	31.0	34.6
170	35.3	38.5

B.5 Comparison with WA

We further compare with workload-aware (WA) [15] separately, as its production trace exposes only hashed inputs with four coarse request types: text, multimodal, file, and search with timestamps. Since hashing prevents intent inference, we replace IAC's 30 intent categories with these four request types and call this variant IAC*. As shown in Fig. 11, IAC* outperforms WA even though both policies group requests identically, showing that IAC's eviction design remains effective even with coarse request-type signals.

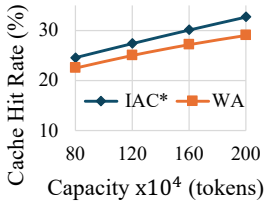
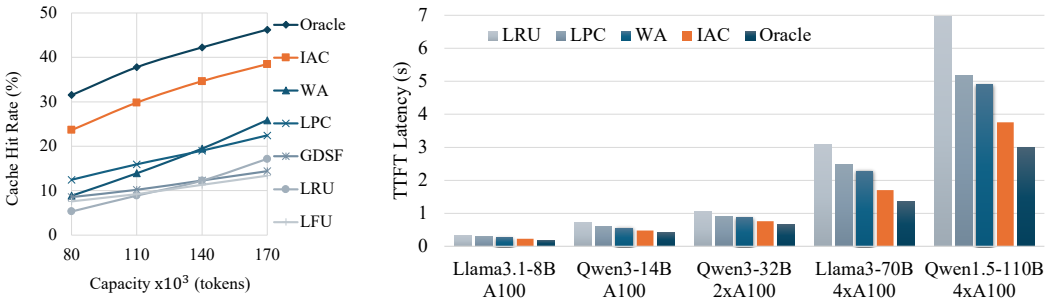


Figure 11: WA [15] trace.



(a) Cache hit rate

(b) TTFT latency

Figure 12: Extended evaluation including WA on ShareChat. (**a**) Cache hit rate across cache capacities. (**b**) TTFT latency across serving settings.

In addition, we test the WA policy on our workloads. WA estimates reuse probability and spatial locality from training data, grouped by request type (text, multimodal, file, and search) and turn index. Since our primary workloads contain only text requests, WA can only use turn-level categories in this setting. In its original block-based cache design for vLLM [11], WA prioritizes each KV block using a tuple based on `ReuseProb` and `Offset`. Here, `ReuseProb` is estimated from the fitted reuse distribution of the request category, while `Offset` gives higher priority to earlier KV blocks to preserve prefix dependencies.

Fig. 12a and Fig. 12b report cache hit rate and TTFT when WA is included in the comparison. The trend is consistent with the main evaluation in Section 5: IAC achieves higher cache hit rate and lower TTFT than WA. For example, on ShareChat at 140K token capacity, IAC reaches 34.6% cache hit rate, compared with 19.4% for WA. This improvement also translates to latency reduction. On Qwen-1.5-110B, IAC reduces TTFT to 3.76 s, compared with 5.90 s for WA. Similarly, on Llama-3-70B, IAC reduces TTFT to 1.7 s, compared with 2.3 s for WA. These results show that even when WA is approximated on our text-only workloads, IAC remains more effective because it uses finer-grained workload categories and delay-aware reuse characterization.

B.6 Robustness to Arrival Rates on LMSys-Chat

To complement the ShareGPT results in Section 5.1 (Fig. 6b), we evaluate IAC on LMSys-Chat [43], which also lacks original timestamps. We construct a synthetic timestamped workload in the same way: new conversations arrive following a Poisson process with rate λ, while inter-turn delays within each conversation are sampled from a long-tailed distribution with mean 4.6 and variance 1, following prior workload studies [39]. The cache capacity is fixed at 150K tokens.

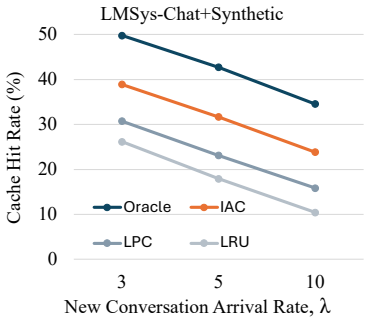


Figure 13: Cache hit rate on LMSys-Chat under different arrival rates λ (150K tokens).

Fig. 13 shows the cache hit rate as λ increases. Higher arrival rates make the workload less cache-friendly for all policies, because more new conversations compete for cache space and force earlier eviction of prefixes from ongoing conversations. IAC remains consistently better than the baselines across all arrival rates. However, its gains are smaller than on the real timestamped workloads (WildChat and ShareChat), because all intent categories share the same synthetic delay distribution. This removes most of the differences in reuse delay across intents and thus limits the benefit of IAC's delay-aware recency signal, leaving the reuse-probability and size signals to drive the improvement.

B.7 Robustness to Arrival Patterns and Uniform Delays

We test bursty and diurnal arrivals on WildChat by sampling the arrival times of new conversations from ON/OFF, Pareto, and diurnal patterns. Table 6 shows that IAC keeps its advantage in all cases,

because its delay-aware signal is learned from inter-turn delays, which the arrival process of new conversations does not change. We further replace all WildChat inter-turn delays with a constant 100 s (Table 7). IAC still far exceeds LRU and stays close to the oracle. With a fixed 100 s delay, every state has $M_{i,t} = 100$ s, so the reuse window is known exactly and the recency test has no estimation error. IAC then orders entries by continuation probability and size, which is enough to track the oracle. A uniform delay therefore does not remove the benefit of IAC. It only shifts the gain from the delay signal to the reuse-probability signal.

Table 6: Cache hit rate (%) on WildChat at 140K token capacity under different arrival patterns of new conversations.

Arrival pattern	LRU	IAC (ours)	Oracle
Diurnal	12.1	19.7	28.5
Pareto	12.5	22.7	28.4
ON/OFF	14.4	22.5	32.2

Table 7: Cache hit rate (%) on WildChat when every inter-turn delay is set to a constant 100 s.

Capacity	LRU	IAC (ours)	Oracle
80×10^3	2.3	9.3	13.0
110×10^3	2.7	12.8	17.2
140×10^3	3.2	16.7	21.4
170×10^3	3.9	20.2	25.1

B.8 Tail Latency

Table 8 extends Fig. 7a with the p50 (median) and p90 TTFT on Llama-3-70B (4×A100) at 10 requests/s. Since IAC raises the cache hit rate and reduces recomputation, it also improves the tail.

Table 8: Mean and tail TTFT (s) on Llama-3-70B (4×A100) at 10 requests/s.

Policy	Mean	p50	p90
LRU	6.51	4.29	17.80
LPC	4.91	3.67	11.84
IAC (ours)	3.43	3.03	6.77
Oracle	3.21	2.07	5.06

B.9 Per-Request Cache Hit Ratio

Table 9 reports the distribution of the per-request cache hit ratio at 16 requests/s and 170K token capacity. The ratio is 0 at the low percentiles under every policy, because single-turn requests and first turns have no cached prefix to reuse. The gains of IAC appear in the upper percentiles, where reuse is possible.

Table 9: Percentiles of the per-request cache hit ratio (%) at 16 requests/s and 170K token capacity. Overall is the token-weighted cache hit rate used in the rest of the paper.

Policy	p10	p25	p50	p75	p90	Overall
LRU	0.0	0.0	0.6	13.3	75.4	17.1
LPC	0.0	0.0	0.7	12.8	74.2	22.4
IAC (ours)	0.0	0.0	0.3	14.3	86.8	38.5
Oracle	0.0	0.0	0.1	7.7	69.9	46.2

C Intent Taxonomy and Labeling Prompt

We present the intent taxonomy used in our workload analysis. We first list the 30-category label space and then present the 20-category and 10-category variants by grouping the original labels under each merged category.

C.1 Thirty Intent Categories

1. `general_explanations_definitions`: Broad factual explanations, concept definitions, or general knowledge questions.

2. `creative_writing`: Creative writing such as stories, poems, scripts, or imaginative text generation.
3. `math_problem_solving`: Solving math questions, equations, symbolic problems, or numerical calculations.
4. `writing_business_marketing`: Business communication, professional writing, ads, pitches, or marketing content.
5. `code_generation`: Writing code from instructions, generating scripts, or implementing programming solutions.
6. `how_to_guides`: Step-by-step instructions for completing a task or learning how to do something.
7. `relationship`: Questions or discussion about romantic or interpersonal relationship issues.
8. `academic_writing`: Academic writing such as essays, reports, research text, or scholarly-style content.
9. `health_medical_info`: Health, symptom, medical, fitness, or wellness-related information requests.
10. `summarization`: Summarizing text, documents, articles, notes, or conversations.
11. `recommendations_products`: Requests for product suggestions, comparisons, or purchase recommendations.
12. `code_debugging`: Finding, explaining, or fixing bugs, errors, or technical issues in code or systems.
13. `chitchat`: Casual conversation, greetings, or light social interaction without a clear task goal.
14. `writing_editing_proofreading`: Revising, proofreading, shortening, polishing, or improving existing text.
15. `data_analysis_statistics`: Statistical analysis, data interpretation, quantitative reasoning, or results explanation.
16. `entertainment_media`: Movies, TV, music, books, games, celebrities, or general entertainment topics.
17. `career_jobs`: Career advice, resume help, interview preparation, job search, or workplace questions.
18. `legal_info`: Questions about laws, legal procedures, rights, or legal concepts.
19. `data_extraction_structuring`: Extracting structured information from unstructured text or reorganizing content into tables, fields, or formats.
20. `translation`: Translating content between languages.
21. `personal_life_advice`: Advice about everyday personal decisions, habits, routines, or general life situations.
22. `mental_wellbeing_support`: Emotional support, stress-related discussion, motivation, or general mental well-being guidance.
23. `research_deep_dive`: In-depth analytical or research-oriented requests requiring detailed explanation or synthesis.
24. `philosophy_religion`: Questions about beliefs, ethics, religion, spirituality, or philosophical ideas.
25. `travel_planning`: Planning trips, itineraries, transportation, destinations, or travel logistics.
26. `education_study_help`: General studying support, concept learning, homework help, or academic tutoring.
27. `personal_finance`: Budgeting, saving, investing, loans, taxes, or other personal finance topics.
28. `food_cooking_recipes`: Recipes, cooking instructions, ingredient substitutions, or food preparation advice.

29. `current_events_politics`: News, politics, public affairs, elections, or recent real-world events.
30. `system_admin_devops`: System setup, deployment, infrastructure, DevOps, cloud, networking, or server management.

C.2 Twenty-Category Taxonomy

The 20-category taxonomy is formed by merging semantically related labels from the 30-category taxonomy. For each merged category below, we list the original labels assigned to it.

1. `advice_support`
   (a) `relationship`
   (b) `personal_life_advice`
   (c) `mental_wellbeing_support`
2. `career_jobs`
   (a) `career_jobs`
3. `chitchat`
   (a) `chitchat`
4. `coding_and_systems`
   (a) `code_generation`
   (b) `code_debugging`
   (c) `system_admin_devops`
5. `current_events_politics`
   (a) `current_events_politics`
6. `education_learning`
   (a) `math_problem_solving`
   (b) `education_study_help`
7. `food_cooking_recipes`
   (a) `food_cooking_recipes`
8. `general_explanations_definitions`
   (a) `general_explanations_definitions`
9. `health_medical_info`
   (a) `health_medical_info`
10. `how_to_guides`
    (a) `how_to_guides`
11. `legal_info`
    (a) `legal_info`
12. `personal_finance`
    (a) `personal_finance`
13. `philosophy_religion`
    (a) `philosophy_religion`
14. `recommendations`
    (a) `recommendations_products`
    (b) `entertainment_media`
15. `research_deep_dive`
    (a) `data_analysis_statistics`
    (b) `data_extraction_structuring`

    (c) research_deep_dive
16. summarization
    (a) summarization
17. translation
    (a) translation
18. travel_planning
    (a) travel_planning
19. writing_creation
    (a) creative_writing
    (b) writing_business_marketing
    (c) academic_writing
20. writing_editing_proofreading
    (a) writing_editing_proofreading

C.3 Ten-Category Taxonomy

The 10-category taxonomy further merges the original labels into broader workload families.

1. advice_support
    (a) relationship
    (b) personal_life_advice
    (c) mental_wellbeing_support
2. career_education
    (a) math_problem_solving
    (b) career_jobs
    (c) education_study_help
3. chitchat
    (a) chitchat
4. coding_debugging
    (a) code_generation
    (b) code_debugging
    (c) system_admin_devops
5. data_work
    (a) data_analysis_statistics
    (b) data_extraction_structuring
6. high_stakes_info
    (a) health_medical_info
    (b) legal_info
    (c) personal_finance
7. how_to_practical
    (a) how_to_guides
    (b) food_cooking_recipes
8. recommendations_planning
    (a) recommendations_products
    (b) entertainment_media
    (c) travel_planning
9. research_news

(a) general_explanations_definitions
(b) research_deep_dive
(c) philosophy_religion
(d) current_events_politics

10. writing_language
    (a) creative_writing
    (b) writing_business_marketing
    (c) academic_writing
    (d) summarization
    (e) writing_editing_proofreading
    (f) translation

C.4 System Prompt for Intent Labeling

System Prompt Used for Intent Labeling

You are an intent classification system for human–LLM conversations. Your task is to assign the **single best intent category** for the **last user message** in the conversation. Choose exactly one category from the list below:

1. chitchat: casual conversation, greetings, or small talk.
2. personal_life_advice: advice on personal decisions, routines, or life situations.
3. relationship_dating: romantic relationships, dating, or interpersonal relationship issues.
4. mental_wellbeing_support: emotional support, stress, motivation, or general mental well-being.
5. health_medical_info: health, symptoms, medical, wellness, or fitness information.
6. legal_info: legal questions, laws, rights, or legal procedures.
7. personal_finance: saving, investing, loans, taxes, budgeting, or money management.
8. career_jobs: jobs, resumes, interviews, workplace, or career advice.
9. education_study_help: studying, tutoring, homework help, or academic learning.
10. math_problem_solving: math problems, equations, or numerical calculations.
11. how_to_guides: step-by-step instructions or practical guidance.
12. recommendations_products: product suggestions, comparisons, or buying advice.
13. travel_planning: trip planning, itineraries, destinations, or travel logistics.
14. food_cooking_recipes: recipes, cooking, ingredients, or food preparation.
15. entertainment_media: movies, music, books, games, celebrities, or media topics.
16. current_events_politics: news, politics, elections, or recent public events.
17. philosophy_religion: ethics, religion, spirituality, or philosophy.
18. general_explanations_definitions: factual explanations, definitions, or broad knowledge questions.
19. research_deep_dive: in-depth research, detailed analysis, or deep explanatory requests.
20. creative_writing: stories, poems, scripts, or imaginative writing.
21. academic_writing: essays, reports, academic drafts, or scholarly writing.
22. writing_business_marketing: business writing, marketing copy, professional messages, or pitches.
23. writing_editing_proofreading: editing, proofreading, rewriting, or improving provided text.
24. summarization: summarizing text, notes, documents, or conversations.

25. `translation`: translating text between languages.
26. `code_generation`: generating code from instructions.
27. `code_debugging`: debugging code, fixing errors, or troubleshooting technical problems.
28. `system_admin_devops`: deployment, infrastructure, servers, cloud, networking, or DevOps operations.
29. `data_analysis_statistics`: data analysis, statistics, interpretation, or quantitative reasoning.
30. `data_extraction_structuring`: extracting structured data or converting text into organized fields or formats.

Instructions:

- Read the full conversation for context.
- Assign the label that best matches the main goal of the last user message.
- If multiple categories seem relevant, choose the one that is most central to the last user request.
- Output only the category name exactly as written above.
- Do not output any explanation.

Examples:

- "Can you write a Python script that reads a CSV and removes duplicate rows?" → `code_generation`
- "My JavaScript code throws undefined is not a function. Can you fix it?" → `code_debugging`
- "Plan a 5-day trip to Japan for me." → `travel_planning`
- "Please shorten and polish this paragraph from my paper." → `writing_editing_proofreading`
- "Explain what overfitting means in machine learning." → `general_explanations_definitions`

Conversation to classify:
`{conversation_text}`